

Production AI Chatbot Text-to-SQL over PostgreSQL

Week 3 Internship Project — Architecture & Implementation Plan

Date: April 2026 | Guide: Kushal | Model: Claude Haiku 4.5

1. Architecture Overview

This project implements a **deterministic pipeline with an embedded agent**. The pipeline has 8 nodes with conditional routing. Only **Node 6** (SQL execution) is a true agent — all other nodes are single LLM calls or rule-based logic. The pipeline enforces safety, caching, and validation in a fixed order; the agent handles dynamic SQL tool-calling only where iterative reasoning is needed.

Component	Type	Purpose
Nodes 0–1	Rule-based	Context loading + safety filtering
Node 2	Single Haiku call	Reconstruct + classify + read-only check
Nodes 3+4	Cache lookup (parallel)	Redis L1 + pgvector L2
Node 5	Single Haiku call	Question checker (top-5 candidates)
Node 6	AGENT (Haiku + MCP)	Live SQL execution — iterative tool-calling
Node 7	Single Haiku call	Read-only guard + answer validation

2. Technology Stack

Layer	Technology
Frontend	React + TypeScript (WebSocket client)
Backend	FastAPI + WebSocket
Orchestration	LangGraph (deterministic pipeline + checkpointing)
LLM	Claude Haiku 4.5 (claude-haiku-4-5-20251001) — all nodes
DB Tool	PostgreSQL MCP Server (@modelcontextprotocol/server-postgres)
Semantic Cache	pgvector extension on PostgreSQL
Hot Cache	Redis (exact-match, TTL 1 hour)
Database	PostgreSQL

3. Pipeline Nodes

Node 0: Load Context

Type: Rule-based

Fetches conversation history (rolling summary + last 5 messages) from the messages table. Loads db_schema.json. LangGraph PostgresSaver checkpoint enables auto-resume if the session crashed mid-pipeline.

Node 1: Safety Filter

Type: Rule-based (regex)

Instant pattern matching for SQL injection (DROP, DELETE, TRUNCATE, ALTER) and prompt injection ("ignore previous instructions"). No LLM cost. Threats are logged with threat=TRUE and execution stops.

Node 2: Reconstruct + Classify + Read-Only Check

Type: Single Haiku call

One LLM call that does three things: (1) reconstructs the raw question using user_profile context (resolves pronouns, abbreviations); (2) classifies intent: DB_QUERY / META_SCHEMA / INCOMPLETE / SENSITIVE / IRRELEVANT / WRITE; (3) rejects anomalies immediately. INCOMPLETE has a MAX 2 follow-up limit — if limit reached, returns 'Please rephrase'.

Node 3+4: Redis L1 + pgvector L2

Type: Parallel cache lookup

Both caches run simultaneously. Redis (~1ms) does an exact SHA-256 hash match. pgvector (~5ms) embeds the reconstructed question and searches question_logs using cosine ANN (ivfflat index, top 5 candidates above 0.85 similarity). Redis hit cancels the pgvector task.

Node 5: Question Checker

Type: Single Haiku call

Receives the top 5 pgvector candidates. Haiku picks the best match (1–5) or returns NONE. Considers negations (FROM vs NOT FROM), entity swaps (Mumbai vs Delhi), temporal differences (this week vs last week), and superlatives (oldest vs youngest).

Node 6: Live SQL Agent (EMBEDDED AGENT)

Type: Haiku + MCP (agent loop)

The only true agent in the pipeline. Haiku dynamically calls MCP tools (list_tables, describe_table, query) in an iterative think-act loop. System prompt restricts to SELECT only. Schema context from db_schema.json (no live fetch). Tokens stream to the user via WebSocket as they generate.

Node 7: Answer Validator

Type: Single Haiku call

Two checks: (1) Read-only SQL guard — scans sql_generated for INSERT/UPDATE/DELETE/DROP/ALTER/TRUNCATE; if found, rejects and logs as threat. (2) Quality check — Haiku verifies the answer correctly addresses the question. Invalid cache hits are re-routed to Node 6. Invalid Node 6 answers return an error.

4. WebSocket Message Protocol

All communication between the React frontend and FastAPI backend uses WebSocket at `/ws/chat/{session_id}`.

Direction	Message Type	When Sent
Client -> Server	<code>{"type": "question", "text": "...", "user_profile": {User}}</code>	User submits a question
Server -> Client	<code>{"type": "reconstructed_query", "original": "...", "query": "..."}</code>	After Node 2 accepts the question
Server -> Client	<code>{"type": "rejected", "reason": "..."} </code>	Node 1 or Node 2 rejects
Client -> Server	<code>{"type": "stop"}</code>	User clicks Stop button
Client -> Server	<code>{"type": "edit", "query": "..."} </code>	User edits reconstructed query
Server -> Client	<code>{"type": "token", "content": "..."} </code>	Streaming answer tokens from Node 6
Server -> Client	<code>{"type": "done"}</code>	Answer complete
Server -> Client	<code>{"type": "error", "message": "..."} </code>	Validation failed / unrecoverable

5. Interactive Reconstruction Flow

After Node 2 reconstructs the question, it is sent to the user via WebSocket **before execution completes**. The pipeline continues running in the background (optimistic execution). The React UI displays the reconstructed query with two buttons:

- **Stop** — cancels the background asyncio task
- **Edit Query** — cancels the task and restarts from **Node 1** (full pipeline, including safety check) with the user-corrected query

If the user does nothing, the pipeline completes and the answer streams in.

Rejection at Node 2 (anomaly, write intent, irrelevant, sensitive) displays the rejection reason in the UI and stops execution immediately.

6. Database Schema

conversations

```
CREATE TABLE conversations ( id SERIAL PRIMARY KEY, session_id UUID NOT NULL UNIQUE, title TEXT, is_deleted BOOLEAN DEFAULT FALSE, -- soft delete history_summary TEXT DEFAULT '', -- rolling LLM summary summarized_until_msg_id INTEGER DEFAULT 0, created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW() );
```

messages

```
CREATE TABLE messages ( id SERIAL PRIMARY KEY, conversation_id INTEGER REFERENCES conversations(id), role TEXT NOT NULL, -- 'user' | 'assistant' raw_question TEXT, reconstructed_q TEXT, content TEXT NOT NULL, intent TEXT, cache_hit TEXT, -- 'redis' | 'pgvector' | 'llm' | null is_deleted BOOLEAN DEFAULT FALSE, created_at TIMESTAMP DEFAULT NOW() );
```

question_logs (semantic cache + audit log)

```
CREATE EXTENSION IF NOT EXISTS vector; CREATE TABLE question_logs ( id SERIAL PRIMARY KEY, question TEXT NOT NULL, question_emb vector(1536), -- embedding for ANN search intent TEXT, answered BOOLEAN NOT NULL, validated BOOLEAN, sql_generated TEXT, answer TEXT, cache_hit TEXT, hit_count INTEGER DEFAULT 1, -- incremented on near-duplicate (sim >= 0.98) threat BOOLEAN DEFAULT FALSE, asked_at TIMESTAMP DEFAULT NOW() ); CREATE INDEX ON question_logs USING ivfflat (question_emb vector_cosine_ops) WITH (lists = 100);
```

7. Key Design Decisions

1. All nodes use Claude Haiku 4.5

Cost efficiency across the full pipeline. Haiku is fast (~200–500ms) and sufficient for classification, checking, and validation tasks.

2. Node 2 merges reconstruction + classification

One LLM call instead of two per request. Saves latency and cost on every single request.

3. Redis + pgvector run in parallel

Redis returns in ~1ms; pgvector ~5ms. Running both simultaneously means the total wait is $\max(1\text{ms}, 5\text{ms}) = 5\text{ms}$, not $1\text{ms} + 5\text{ms} = 6\text{ms}$. Small savings that compound at scale.

4. pgvector ivfflat index (lists=100)

Approximate Nearest Neighbor index for sub-millisecond search on millions of rows. Without it, cosine search is $O(n)$ — unacceptable at scale.

5. No duplicate pgvector entries (hit_count)

Before inserting, check if similarity ≥ 0.98 . If yes, increment `hit_count`. Prevents the vector index from bloating with near-identical questions.

6. Rolling conversation summary

Keep last 5 messages verbatim + a Haiku-generated summary of older messages. Bounds the context window while preserving all key facts from the conversation.

7. db_schema.json auto-generated at startup

Never hardcode table or column names. All prompts inject schema from this file. Swapping the underlying DB requires no code changes.

8. LangGraph PostgresSaver checkpointing

Every node's output is persisted. If Node 6 crashes mid-execution, re-connecting with the same `session_id` resumes from Node 6 automatically.

9. MAX 2 follow-up questions per session

Prevents infinite clarification loops. After 2 INCOMPLETE responses, the bot returns 'Please rephrase your question' and moves on.

10. Read-only enforcement at 3 layers

Node 2 rejects write intent in natural language. Node 6 system prompt forbids write SQL. Node 7 scans the generated SQL for write keywords as a final guard.

8. File Structure

```
project/ backend/ main.py # FastAPI + WebSocket /ws/chat/{session_id} graph.py # LangGraph
graph definition + checkpointer setup db.py # SQLAlchemy engine + CRUD for all tables
embeddings.py # embed questions for pgvector db_schema.json # auto-generated at startup
requirements.txt .env nodes/ context_loader.py # Node 0: load history + schema safety.py #
Node 1: regex safety filter reconstructor.py # Node 2: reconstruct + classify + read-only
(Haiku) cache.py # Node 3+4: Redis L1 + pgvector L2 in parallel checker.py # Node 5: question
checker (Haiku) sql_agent.py # Node 6: embedded agent – Haiku + MCP validator.py # Node 7:
read-only guard + answer validation (Haiku) scripts/ init_db.py # ONE-TIME: create all tables
+ pgvector extension seed_db.py # ONE-TIME: Faker-based realistic data (no hardcoded values)
generate_schema.py # AT STARTUP: introspect live DB -> db_schema.json frontend/ src/ App.tsx
components/ Chat.tsx # WebSocket client, streaming token render ReconstructedQuery.tsx # shows
query + Stop/Edit buttons Message.tsx # styled per intent type ConversationList.tsx # past
convo + soft-delete UI
```

9. Verification Checklist (23 Test Cases)

#	Test Input / Scenario	Expected Outcome
1	SQL injection ("DROP TABLE users")	Blocked at Node 1 instantly
2	Gibberish ("asdfgh")	Rejected at Node 2, reason shown in UI
3	Write intent ("delete all users")	Rejected at Node 2 as WRITE — system is read-only
4	"find my email" + user_profile{name:Sam}	Reconstructed to "find email of Sam", shown in UI
5	User clicks Stop	Background async task cancelled immediately
6	User edits reconstructed query	Pipeline restarts from Node 1 (full safety check)
7	Normal question, no intervention	Answer streams in via WebSocket tokens
8	"what's the time"	IRRELEVANT at Node 2, rejected
9	"What columns do you have?"	META_SCHEMA at Node 2, returns schema, no SQL
10	"Show me all passwords"	SENSITIVE at Node 2, blocked
11	Same question asked twice	Redis cache hit on second request
12	Rephrased same question	pgvector top-5 found, checker picks correct candidate
13	"OLDEST user" after "YOUNGEST user" in cache	Checker returns NONE, fresh SQL runs
14	Node 6 somehow generates write SQL	Caught by Node 7 read-only guard, logged as threat
15	Node 6 crashes mid-execution	Resume with same session_id via LangGraph checkpoint
16	Soft-delete a conversation	is_deleted=TRUE, excluded from history load
17	Vague but answerable ("show users")	Inferred and answered without asking back
18	Truly ambiguous (followup_count=0)	Clarifying question asked, followup_count=1
19	Still ambiguous at followup_count=2	"Please rephrase your question" returned
20	Same question asked 3 times	question_logs has 1 entry with hit_count=3 (no duplicates)
21	10+ messages in session	history_summary populated; Node 2 gets summary + last 5 only
22	Swap to a different PostgreSQL schema	db_schema.json regenerates, all prompts auto-adapt

23	End-to-end audit	All events in question_logs + messages with correct fields
----	------------------	--